application can display a helpful message: “Access to team reports is restricted outside of working hours.”

To Alice, the system simply told her no. However, behind the scenes, that decision resulted from a carefully orchestrated, policy-driven process involving the PEP, PDP, and PIP—each playing a clear, well-defined role.

This flow—intercept, evaluate, enrich, decide, enforce—lies at the heart of any effective, policy-driven authorization system. It transforms identity and context into trustworthy, real-time decisions on which systems can rely.

4.4.2 Where policies come from

Meet Carol. She’s a security administrator at ACME Corp, responsible for defining and managing access policies across the company’s internal systems. While Alice, a project manager at ACME, reviews team dashboards and updates project timelines, Carol works behind the scenes to ensure that only the right people can do the right things under the right conditions.

Carol doesn’t embed authorization logic into the application code. Instead, she writes policies in a centralized management system—the Policy Administration Point (PAP) as shown in figure 4.2. Here, policies are created, reviewed, versioned, and deployed much like code. It’s also where policy changes can be tested and rolled out using familiar software development workflows.

Let’s say Carol wants to ensure that only users in the Project Managers group can access the Team Reports section of ACMEProject—and only during working hours. She writes a policy that captures this logic in a structured, declarative format using a language such as Cedar. Here’s the policy Carol defined that denied Carol access outside work hours:

```
permit(
    principal in Group::"ProjectManagers",
    action == Action::"view",
    resource in Report::"TeamReports"
) when {
    context.time >= "07:00" && context.time <= "20:00"
};
```